

Credit Card Fraud Detection Using Machine Learning Classifiers



Indian Statistical Institute(ISI), Bangalore

Siddhesh Deepak Rane
Guided By: **Dr. Rituparna Sen**

Abstract

This study investigates the efficacy of various machine learning classifiers in detecting credit card fraud, with a focus on maintaining high accuracy while minimising false positives. A highly imbalanced dataset comprising anonymised transaction records was utilised, and nine classification models—including XGBoost, Random Forest, Decision Tree, K-Nearest Neighbours, Logistic Regression, Linear Discriminant Analysis, Support Vector Machine, Gaussian Naive Bayes, and AdaBoost—were evaluated. Performance was assessed using standard metrics such as precision, recall, F1-score, accuracy, AUC, and confusion matrices. To address class imbalance, three data balancing techniques were applied: Random Oversampling, Random Undersampling, and the Synthetic Minority Oversampling Technique (SMOTE). The results indicate that Random Oversampling in combination with XGBoost or Random Forest yields the best overall performance, achieving a favourable balance between fraud detection and minimisation of false positives. This study highlights the importance of both robust model selection and appropriate data balancing in the development of effective fraud detection systems.

Contents

Contents	2
1 Introduction	3
2 Literature Review	3
2.1 Description of the Models	4
2.1.1 XGBoost	4
2.1.2 Random Forest	4
2.1.3 Decision Tree	4
2.1.4 K-Nearest Neighbours (KNN)	5
2.1.5 Logistic Regression	6
2.1.6 Linear Discriminant Analysis	7
2.1.7 Support Vector Machine	7
2.1.8 Gaussian Naive Bayes	8
2.1.9 AdaBoost Classifier	9
2.2 Data Balancing Methods	9
2.2.1 Random Oversampling	9
2.2.2 Random Undersampling	10
2.2.3 Synthetic Minority Oversampling Technique (SMOTE)	10
2.3 Evaluation Metrics	11
2.3.1 Precision	11
2.3.2 Recall	11
2.3.3 Accuracy	12
2.3.4 F1 Score	12
2.3.5 Area Under the Curve (AUC)	12
2.3.6 Confusion Matrix	13
3 Description of Data	14
4 Methodology	14
5 Results	15
References	22

1 Introduction

With the rising number of individuals using credit cards, incidents of credit card fraud have also increased significantly. It has, therefore, become essential for financial institutions to detect fraudulent transactions swiftly while maintaining a low rate of false positives. Minimising false alarms not only conserves operational resources but also helps preserve customer trust. This study aims to identify the most effective classification model for fraud detection by comparing the performance of several algorithms — XGBoost, Random Forest, Decision Tree, and K-Nearest Neighbours, Support Vector Machine, Logistic Regression, Gaussian Naive Bayes, AdaBoost, Linear Discriminant Analysis, — using the evaluation metrics: accuracy, precision, recall, and F1 score, Area Under the ROC Curve(AUC), Confusion matrix.

2 Literature Review

Supervised Learning

Supervised learning is a fundamental concept in machine learning in which models are trained using labelled data. Each instance in the training dataset consists of an input-output pair, where the input represents a set of features and the output corresponds to the known target label or value. The learning algorithm analyses these labelled examples to identify patterns and relationships that can generalise to unseen data.

Once the model has been trained, it can be used to predict outputs for new, unlabelled inputs. Supervised learning encompasses both classification and regression tasks: classification involves predicting discrete class labels, while regression aims to predict continuous numerical values. The effectiveness of supervised learning depends heavily on the quality and representativeness of the labelled training data, as well as the appropriateness of the chosen model and evaluation metrics.

Classification

Classification is a type of supervised learning task where the objective is to predict a discrete or categorical output label for a given input instance. In this setting, the model is trained on a dataset comprising input features paired with known class labels. The goal is to learn a decision function that can accurately assign a category to new, unseen inputs based on patterns identified during training.

Classification algorithms are widely used in applications such as fraud detection, spam filtering, medical diagnosis, and image recognition. Common classification models include Logistic Regression, Decision Trees, Random Forest, Support Vector Machines, and K-Nearest Neighbours. The performance of classification models is typically evaluated using metrics such as accuracy, precision, recall, F1-score, and AUC-ROC, particularly in situations where class distributions are imbalanced.

2.1 Description of the Models

2.1.1 XGBoost

Gradient Boosting is an ensemble learning technique that constructs models sequentially by minimising a specified loss function through gradient descent. At each iteration, the algorithm fits a decision tree to the negative gradients of the loss function with respect to the current model's predictions. These gradients indicate the direction of steepest descent required to reduce the loss. The newly fitted tree is then incorporated into the existing model, typically scaled by a learning rate to mitigate the risk of overfitting. This iterative process continues until the loss function converges or a predefined number of iterations is reached.

XGBoost (Extreme Gradient Boosting) is a highly optimised implementation of this approach, designed to enhance computational speed and predictive performance. In addition to its efficient tree-building procedures, XGBoost includes built-in regularisation techniques—specifically L1 and L2 penalties—to further prevent overfitting and improve generalisation.

2.1.2 Random Forest

Random Forest is an ensemble learning method that operates by constructing a multitude of decision trees and aggregating their outputs to improve prediction accuracy and control overfitting.

Given a dataset with a feature space consisting of n variables, the algorithm first creates b bootstrap samples—denoted $D_1, D_2, \dots, D_b$ —by randomly sampling with replacement from the original dataset. For each bootstrap sample, a subset of p features (where $p < n$) is randomly selected without replacement, ensuring diversity among the trees. A separate decision tree T_i is then trained on each dataset D_i , resulting in a collection of trees $T_1, T_2, \dots, T_b$, collectively forming the *Random Forest*.

$$D_1, D_2, \dots, D_b \Rightarrow T_1, T_2, \dots, T_b = \mathcal{T}$$

The final prediction is obtained by aggregating the outputs of all trees in the ensemble. For classification tasks, this is typically done via a majority vote across all trees, while for regression problems, the predictions are averaged. The incorporation of both bootstrap sampling and random feature selection introduces variance reduction and decorrelation among trees, which enhances the generalisation capability of the model. As a result, Random Forest is robust to overfitting, particularly when compared to single decision trees, and performs well on a wide range of classification and regression tasks.

2.1.3 Decision Tree

Decision Trees are supervised learning algorithms used for both classification and regression tasks. They operate by recursively partitioning the feature space into increasingly homogeneous subsets based on a series of decision rules. At each node of the tree, the algorithm evaluates all possible splits across available features and selects the feature

and corresponding threshold that yield the greatest reduction in a chosen impurity measure. For classification problems, common impurity measures include *Gini impurity* and *entropy*, while for regression tasks, criteria such as *mean squared error (MSE)* or *mean absolute error (MAE)* are often used.

Once the optimal split is determined, the dataset is divided into two child nodes, and the process is repeated recursively on each subset. This hierarchical partitioning continues until a stopping condition is met—such as reaching a maximum tree depth, achieving pure leaf nodes, or having too few samples to split further. The resulting structure resembles a flowchart, where internal nodes represent decision points, branches correspond to outcomes of those decisions, and leaf nodes contain final predictions. Despite their interpretability and simplicity, decision trees are prone to overfitting, particularly on noisy datasets, which suggests the use of ensemble methods such as Random Forest or Gradient Boosting to improve generalisation.

For classification, we define:

$$p_{mk} = \frac{1}{N_m} \sum_{x_i \in R_m} I(y_i = k)$$

The predicted class $k(m)$ is then given by:

$$k(m) = \arg \max_k p_{mk}$$

For regression, the prediction for region R_m is:

$$k(m) = \frac{1}{N_m} \sum_{x_i \in R_m} y_i$$

where $k(m)$ is the class assigned to region R_m as the value of k that maximises p_{mk} .

2.1.4 K-Nearest Neighbours (KNN)

K-Nearest Neighbours (KNN) is a non-parametric, instance-based learning algorithm commonly used for classification and regression tasks. It is referred to as a lazy learner because it does not construct an explicit predictive model during the training phase. Instead, it retains the entire training dataset and defers computation until a prediction is required. When classifying a new data point, the algorithm identifies the K instances in the training set that are most similar to the query point, typically using a distance metric such as *Euclidean distance*, *Manhattan distance*, or *Minkowski distance*.

Once the K nearest neighbours are identified, the algorithm determines the predicted class by taking the *mode* (most frequent class label) among those neighbours in classification tasks. In the case of regression, the prediction is usually computed as the *mean* of the target values of the K nearest neighbours. The choice of K significantly influences model performance: a small K may lead to overf

2.1.5 Logistic Regression

Logistic Regression is a widely used statistical model for binary classification tasks, it can, however, be extended to multiclass problems through techniques such as multinomial logistic regression. Logistic regression is a classification algorithm rather than a regression method, as it predicts discrete outcomes by modelling the probability that a given input belongs to a particular class.

The model estimates the probability of the positive class using the logistic (sigmoid) function, which maps real-valued inputs to the interval $(0, 1)$:

$$P(y = 1 \mid \mathbf{x}) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}}$$

where $\mathbf{x}$ is the input feature vector, $\mathbf{w}$ represents the vector of learned weights, and b is a constant term. The predicted probability is then required to pass the threshold (commonly at 0.5) to assign a class label.

Model training is performed by maximising the likelihood of the observed data. This is typically achieved using optimisation techniques like gradient descent.

Logistic regression assumes a linear relationship between the input features and the log-odds of the outcome, making it easy to interpret and computationally efficient. However, its performance may degrade when applied to datasets with complex, non-linear relationships..

2.1.6 Linear Discriminant Analysis

Linear Discriminant Analysis (LDA) is a classification method that differs fundamentally from discriminative models such as logistic regression. While logistic regression directly models the conditional probability $P(y \mid \mathbf{x})$, LDA instead models the class-conditional densities $P(\mathbf{x} \mid y = k) = f_k(\mathbf{x})$ and the class prior probabilities $P(y = k) = q_k$. Using Bayes' Theorem, the posterior probability of a data point $\mathbf{x}$ belonging to class k is given by:

$$P(y = k \mid \mathbf{x}) = \frac{q_k \cdot f_k(\mathbf{x})}{\sum_i q_i \cdot f_i(\mathbf{x})} = \frac{q_k \cdot f_k(\mathbf{x})}{m(\mathbf{x})}$$

where $m(\mathbf{x})$ denotes the marginal probability distribution of the input data. In both *Linear* and *Quadratic Discriminant Analysis* (QDA), the class-conditional distributions $f_k(\mathbf{x})$ are modelled using Gaussian distributions.

LDA not only provides probabilistic classification but also performs dimensionality reduction by projecting the data onto a lower-dimensional subspace that maximises class separability. It is particularly effective when the assumptions of normally distributed features and equal class covariances hold, and it remains a valuable method in scenarios where interpretability and computational efficiency are important.

2.1.7 Support Vector Machine

Support Vector Machines (SVM) are powerful supervised learning models used primarily for classification tasks, although they can also be adapted for regression. The fundamental idea behind SVM is to identify an optimal hyperplane that separates the data points of different classes with the maximum possible margin. The *margin* is defined as the distance between the separating hyperplane (also known as the decision boundary) and the nearest data points from each class. These closest data points are referred to as *support vectors*, and they play a critical role in determining the orientation and position of the hyperplane.

SVM aims to maximise this margin to improve the generalisability of the classifier. In linearly separable cases, the algorithm identifies a hyperplane that perfectly divides the classes. However, in practice, some data points may lie within the margin or even on the wrong side of the boundary. To handle such situations, the *soft margin* SVM introduces slack variables that allow for a certain degree of misclassification, thereby achieving a balance between margin maximisation and classification error.

2.1.8 Gaussian Naive Bayes

Naive Bayes is a supervised learning algorithm that is primarily employed for *classification tasks*. It functions by estimating the probability that a given instance belongs to a particular class, based on the observed values of its features.

To classify a new instance, Naive Bayes evaluates the *conditional probability*:

$$P(y = \text{class}_i \mid x = (x_1, x_2, \dots, x_n)),$$

where x denotes the vector of features corresponding to the instance. The class associated with the *highest conditional probability* is then assigned to the instance.

This probability is computed using Bayes' Theorem, which states:

$$P(y \mid x) = \frac{P(x \mid y) \cdot P(y)}{P(x)}.$$

Naive Bayes introduces the simplifying assumption that all features x_i are *conditionally independent* given the class label y . That is,

$$P(x \mid y) = \prod_{i=1}^n P(x_i \mid y).$$

Substituting this into Bayes' Theorem yields:

$$P(y \mid x) = \frac{P(y) \cdot \prod_{i=1}^n P(x_i \mid y)}{P(x)}.$$

In practical applications, the denominator $P(x)$ is constant across all classes and can therefore be omitted for the purposes of classification. This leads to the simplified decision rule:

$$P(y \mid x) \propto P(y) \cdot \prod_{i=1}^n P(x_i \mid y).$$

The choice of distribution for each $P(x_i \mid y)$ depends on the nature of the data. In the case of Gaussian Naive Bayes, it is assumed that each feature x_i , conditioned on the class y , follows a *Gaussian (normal) distribution*. The parameters of this distribution — specifically the mean and variance — are estimated from the training data for each class-feature combination.

2.1.9 AdaBoost Classifier

AdaBoost is an ensemble learning method primarily used for *classification tasks*. It constructs a strong classifier by combining multiple *weak learners*, each of which may perform only slightly better than random guessing. The key principle of AdaBoost lies in its ability to *adaptively adjust the weights* of training examples so that subsequent weak learners focus more on the *difficult cases*, i.e., those that were misclassified by previous learners. Initially, all training instances are assigned *equal weights*. A weak learner is then trained on this weighted dataset. After training, the performance of the learner is evaluated, and the *weights of the training examples are updated* based on the classification outcomes. Specifically, the weights of *misclassified* instances are *increased*, while the weights of *correctly classified* instances are *decreased*. This ensures that the next weak learner pays greater attention to the more challenging examples.

The updated, reweighted dataset is then used to train the next weak learner. This process is repeated iteratively, with each new learner being trained on a dataset that reflects the errors of its predecessors. The final model is a *weighted combination* of all the weak learners, where each learner's contribution is determined by its classification accuracy.

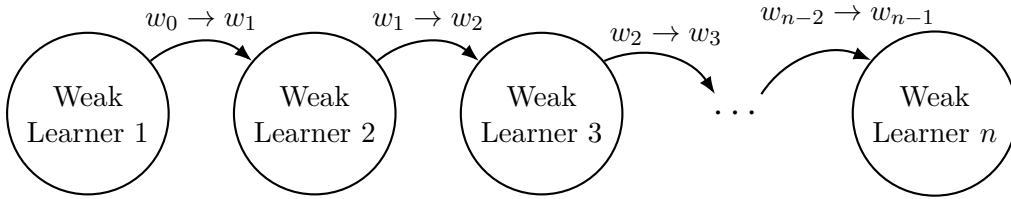


Figure 1: Illustration of AdaBoost's sequential training: each weak learner updates instance weights to focus on harder cases, passing them forward to the next learner.

2.2 Data Balancing Methods

Due to the substantial class imbalance present in the original dataset, model performance may be biased in favour of the majority class, resulting in poor detection of minority class instances—such as fraudulent transactions in a fraud detection context. This imbalance can lead to misleading accuracy metrics and reduced generalisability, particularly in critical applications where minority class detection is essential.

To construct a more robust and reliable model, the training data may be pre-processed in a manner that achieves a more balanced class distribution. This involves adjusting the number of observations in each class so that the classifier does not favour the majority class during training. Several data balancing techniques are available to address this issue, each with different strategies for adjusting the class proportions. The methods employed in this study are outlined below.

2.2.1 Random Oversampling

Random Oversampling is a straightforward technique used to address class imbalance by increasing the representation of the minority class within the training dataset. This is achieved by randomly duplicating existing instances of the minority class until the

desired class balance is attained. Unlike synthetic data generation methods, random oversampling does not introduce new or artificial examples; it simply replicates original observations.

While this approach is effective in reducing class bias and improving the classifier’s ability to recognise minority class patterns, it does not introduce additional variability in the data. As a result, it may increase the likelihood of overfitting, particularly when the same minority instances are repeatedly sampled. Yet, due to its simplicity and ease of implementation, random oversampling remains a widely used baseline technique for handling imbalanced datasets.

2.2.2 Random Undersampling

Random Undersampling is a technique employed to mitigate class imbalance by reducing the number of samples from the majority class. This method involves randomly removing instances from the majority class until a more balanced class distribution is achieved in the training dataset. By doing so, it helps prevent the learning algorithm from being biased towards the overrepresented class, thereby improving the model’s ability to recognise patterns in the minority class.

However, a key limitation of random undersampling is the potential loss of valuable information, as informative samples may be inadvertently discarded during the selection process. This loss can lead to a decrease in model performance, particularly if the remaining majority class samples do not sufficiently capture the underlying distribution. Despite this drawback, random undersampling remains a commonly used and computationally efficient method for rebalancing datasets, especially when the dataset is large and the majority class is significantly overrepresented.

2.2.3 Synthetic Minority Oversampling Technique (SMOTE)

SMOTE is an advanced oversampling technique designed to address class imbalance by generating synthetic instances of the minority class, rather than simply duplicating existing samples. Unlike random oversampling, SMOTE increases the diversity of the minority class by introducing new, artificial data points that are interpolated between existing ones.

The algorithm operates by identifying the k -nearest neighbours of each instance in the minority class based on a distance metric, commonly Euclidean distance. It then selects one or more of these neighbours at random and generates a new synthetic sample by interpolating between the feature values of the original instance and its selected neighbour. This process is repeated until the desired level of class balance is achieved.

By enriching the minority class with more varied examples, SMOTE can improve a classifier’s ability to generalise, particularly when the minority class is underrepresented. However, care must be taken when applying SMOTE to datasets with overlapping class

distributions, as synthetic examples may be introduced in regions where class boundaries are unclear, potentially leading to increased misclassification.

2.3 Evaluation Metrics

The choice of evaluation metrics plays a crucial role in assessing the performance of machine learning algorithms, particularly when dealing with imbalanced datasets. Metrics determine how well a model distinguishes between classes and influence the interpretation and comparison of different algorithms. While accuracy is a commonly used metric, it may be misleading in scenarios where one class significantly outweighs the other, as it can mask poor performance on the minority class.

To obtain a more reliable and nuanced understanding of model effectiveness, additional metrics such as precision, recall, F1-score, and the area under the receiver operating characteristic curve (AUC-ROC) are employed. These metrics provide deeper insight into how well a model captures true positives and avoids false positives. Choosing appropriate evaluation metrics is therefore essential to ensure meaningful comparisons and to guide the selection of the most suitable model for the task at hand.

2.3.1 Precision

Precision is an evaluation metric that measures the proportion of correctly predicted positive instances out of all instances predicted as positive by the model. It is formally defined as:

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

Precision is particularly important in scenarios where the cost of false positives is high. For example, in fraud detection, incorrectly classifying a legitimate transaction as fraudulent (a false positive) could lead to unnecessary customer inconvenience and loss of trust. A high precision score indicates that the classifier has a low rate of false positive predictions, making it a suitable metric when false alarms carry significant consequences. However, precision alone does not account for missed positive cases (false negatives).

2.3.2 Recall

Recall, also known as *sensitivity* or *true positive rate*, is an evaluation metric that measures the proportion of actual positive instances that are correctly identified by the model. It is formally defined as:

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

Recall is particularly important in situations where the cost of false negatives is high. For example, in credit card fraud detection, failing to identify a fraudulent transaction (a false negative) may result in financial loss and reputational damage. A high recall score

indicates that the model successfully detects most of the actual positive cases, even if it means accepting more false positives. However, focusing solely on recall may compromise precision, which is why it is often evaluated in conjunction with other metrics such as precision or the F1-score.

2.3.3 Accuracy

Accuracy is a fundamental evaluation metric that measures the overall proportion of correct predictions made by a classification model. It is defined as the ratio of the number of correct predictions to the total number of predictions:

$$\text{Accuracy} = \frac{\text{True Positives} + \text{True Negatives}}{\text{Total Predictions}}$$

Accuracy provides a straightforward measure of model performance and is often used as a baseline metric. However, it is most appropriate when the dataset has a balanced class distribution—that is, when each class is represented by a roughly equal number of instances. In cases of class imbalance, accuracy can be misleading, as a model may achieve high accuracy by predominantly predicting the majority class while failing to correctly identify minority class instances. Consequently, in imbalanced settings, complementary metrics such as precision, recall, and F1-score are preferred for a more comprehensive evaluation.

2.3.4 F1 Score

F1-score is an evaluation metric that captures the balance between precision and recall, offering a single measure that accounts for both false positives and false negatives. It is defined as the harmonic mean of precision and recall:

$$\text{F1-score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Unlike the arithmetic mean, the harmonic mean penalises extreme values, making the F1-score particularly useful when there is an uneven trade-off between precision and recall. This metric is especially well-suited for imbalanced datasets, where focusing on either precision or recall in isolation may give an incomplete picture of model performance. A high F1-score indicates that both false positives and false negatives are being minimised effectively, making it a preferred choice in domains such as fraud detection, medical diagnostics, and information retrieval.

2.3.5 Area Under the Curve (AUC)

AUC is an evaluation metric commonly used in binary classification problems. It represents the area under the ROC (Receiver Operating Characteristic) curve, which illustrates the model's performance across all possible classification thresholds.

The ROC curve plots the False Positive Rate (FPR) on the X-axis against the True Positive Rate (TPR) on the Y-axis. As the classification threshold varies from 0 to 1, both FPR and TPR change, tracing out the ROC curve.

As the threshold decreases, the model becomes more likely to predict the positive class. This results in an increase in both TPR and FPR — the model correctly identifies more positives, but also risks misclassifying more negatives as positives.

The AUC score quantifies this trade-off. An AUC of 1 indicates a perfect model — one that achieves a high TPR even when the FPR is very low, meaning it correctly identifies true positives without mistakenly labelling negatives as positives. An AUC of 0.5 corresponds to a model performing no better than random guessing, while an AUC of 0 would reflect a model that consistently predicts the opposite of the correct class.

Thus, higher AUC values indicate better model performance, particularly in distinguishing between the positive and negative classes.

2.3.6 Confusion Matrix

The confusion matrix is a fundamental tool for evaluating the performance of a classification model, particularly in binary classification tasks. In the binary case, the confusion matrix is typically represented as a 2×2 table, structured as follows:

	Predicted Negative	Predicted Positive
Actual Negative	True Negative (TN)	False Positive (FP)
Actual Positive	False Negative (FN)	True Positive (TP)

- **True Positives (TP):** Instances that are correctly predicted as positive.
- **False Positives (FP):** Instances that are predicted as positive but are actually negative.
- **True Negatives (TN):** Instances that are correctly predicted as negative.
- **False Negatives (FN):** Instances that are predicted as negative but are actually positive.

The confusion matrix provides a detailed breakdown of a model's predictions and facilitates the calculation of key evaluation metrics, such as: Accuracy, Precision, Recall, F1 Score

3 Description of Data

The dataset used in this study consists of credit card transactions carried out by European cardholders during September 2013. It comprises a total of 284,807 transactions recorded over a period of two days. Of these, only 492 transactions (approximately 0.172%) have been labelled as fraudulent, making the dataset highly imbalanced.

To maintain the privacy of the individuals and institutions involved—and due to confidentiality constraints—the original raw data has not been made publicly available. Instead, the dataset released for research purposes includes only numerical input variables that have undergone a principal component analysis (PCA) transformation. Specifically, the features labelled **V1**, **V2**, ..., **V28** represent the principal components obtained through PCA, which aim to preserve important variance in the data while concealing sensitive information.

In addition to these anonymised components, two original features are retained without transformation. The first is **Time**, which indicates the number of seconds elapsed between each transaction and the first transaction in the dataset. The second is **Amount**, which represents the monetary value of the transaction. The binary **Class** variable serves as the target, where a value of 1 indicates a fraudulent transaction and 0 indicates a legitimate one. [1].

4 Methodology

The initial phase of the analysis involves data preprocessing to ensure quality and consistency. All instances containing missing values are removed from the dataset, resulting in a cleaned version suitable for model training and evaluation. The cleaned data is then partitioned into training and test subsets(80:20).

Each classification algorithm is first trained on the unbalanced training data, and its performance is evaluated on the test set using standard evaluation metrics, including accuracy, precision, recall, F1-score and AUC. Given the significant class imbalance in the original dataset, this step serves as a baseline to highlight the challenges associated with learning from imbalanced data.

To mitigate the effects of class imbalance, various data balancing techniques are subsequently applied to the training data. These include *Random Oversampling*, *Random Undersampling*, and *SMOTE* (Synthetic Minority Over-sampling Technique). After applying each technique, the classification models are retrained using the newly balanced datasets, and their performance is re-evaluated using the same set of metrics.

The objective of this study is to identify the classification model that performs most effectively in detecting fraudulent transactions while assessing the impact of each data balancing technique on model performance. The model achieving the highest scores across the evaluation metrics is deemed the most suitable for the fraud detection task.

5 Results

The study utilised six evaluation metrics to find the best model. The findings are listed below:

Confusion Matrix

XGBoost

Unbalanced		Oversampling	
56857	7	56854	10
19	79	13	85

Undersampling		SMOTE	
54255	2609	56833	31
8	90	15	83

Random Forest

Unbalanced		Oversampling	
56859	5	56861	3
18	80	21	77

Undersampling		SMOTE	
54789	2075	56852	12
9	89	20	78

Model: Decision Tree

Unbalanced

56855	9
22	76

Oversampling

56836	28
29	69

Undersampling

54018	2846
11	87

SMOTE

56746	118
21	77

KNN

Unbalanced

56859	5
79	19

Oversampling

56084	780
10	88

Undersampling

54821	2043
9	89

SMOTE

55480	1384
8	90

Logistic Regression

Unbalanced

56851	13
35	63

Oversampling

55486	1378
8	90

Undersampling

54553	2311
8	90

SMOTE

55399	1465
8	90

LDA

Unbalanced

56847	17
19	79

Oversampling

56080	784
15	83

Undersampling

55964	900
16	82

SMOTE

56078	786
13	85

SVM

Unbalanced

56849	15
26	72

Oversampling

55212	1652
17	81

Undersampling

50025	6839
8	90

SMOTE

55250	1614
17	81

Gaussian NB

Unbalanced

55535	1329
15	83

Oversampling

55303	1561
12	86

Undersampling

54647	2217
12	86

SMOTE

55418	1446
12	86

AdaBoost

Unbalanced		Oversampling	
56834	30	55662	1202
26	72	11	87

Undersampling		SMOTE	
54638	2226	55193	1671
8	90	9	89

It is observed that false positives increase uniformly and disproportionately across all models when Random Undersampling is applied.

Unbalanced Evaluation

Model	Accuracy	Precision	Recall	F1 Score	AUC
XGBoost	0.99954	0.91860	0.80612	0.85870	0.97432
Random Forest	0.99960	0.94118	0.81633	0.87432	0.95332
Decision Tree	0.99946	0.89412	0.77551	0.83060	0.90293
KNN	0.99853	0.79167	0.19388	0.31148	0.83632
Logistic Regression	0.99916	0.82895	0.64286	0.72414	0.95747
LDA	0.99937	0.82292	0.80612	0.81443	0.98107
SVM	0.99928	0.82759	0.73469	0.77838	0.92368
Gaussian NB	0.97641	0.05878	0.84694	0.10993	0.96325
AdaBoost	0.99902	0.70588	0.73469	0.72000	0.97923

For the unbalanced dataset, the Decision Tree model appears to perform the best, followed closely by Linear Discriminant Analysis (LDA).

Balanced Evaluation

Random Oversampling

Model	Accuracy	Precision	Recall	F1 Score	AUC
XGBoost	0.99960	0.89474	0.86735	0.88083	0.97680
Random Forest	0.99958	0.96250	0.78571	0.86517	0.95826
Decision Tree	0.99900	0.71134	0.70408	0.70769	0.85179
KNN	0.98613	0.10138	0.89796	0.18219	0.94961
Logistic Regression	0.97567	0.06131	0.91837	0.11494	0.97202
LDA	0.98597	0.09573	0.84694	0.17202	0.96048
SVM	0.97070	0.04674	0.82653	0.08848	0.91817
Gaussian NB	0.97239	0.05222	0.87755	0.09857	0.96300
AdaBoost	0.97871	0.06749	0.88776	0.12545	0.96964

Random Undersampling

Model	Accuracy	Precision	Recall	F1 Score	AUC
XGBoost	0.95406	0.03335	0.91837	0.06435	0.97486
Random Forest	0.96341	0.04113	0.90816	0.07869	0.97743
Decision Tree	0.94984	0.02966	0.88776	0.05741	0.91115
KNN	0.96398	0.04174	0.90816	0.07982	0.96239
Logistic Regression	0.95929	0.03748	0.91837	0.07203	0.97671
LDA	0.98392	0.08350	0.83673	0.15185	0.96443
SVM	0.87980	0.01299	0.91837	0.02562	0.96418
Gaussian NB	0.96087	0.03734	0.87755	0.07164	0.95877
AdaBoost	0.96078	0.03886	0.91837	0.07457	0.97231

SMOTE

Model	Accuracy	Precision	Recall	F1 Score	AUC
XGBoost	0.99919	0.72807	0.84694	0.78302	0.97998
Random Forest	0.99944	0.86667	0.79592	0.82979	0.95512
Decision Tree	0.99756	0.39487	0.78571	0.52560	0.89182
KNN	0.97556	0.06106	0.91837	0.11450	0.95205
Logistic Regression	0.97414	0.05788	0.91837	0.10889	0.97090
LDA	0.98597	0.09759	0.86735	0.17544	0.95587
SVM	0.97137	0.04779	0.82653	0.09035	0.91842
Gaussian NB	0.97440	0.05614	0.87755	0.10552	0.96458
AdaBoost	0.97051	0.05057	0.90816	0.09580	0.97371

For balanced datasets, Random Undersampling consistently yields the poorest performance, demonstrating the lowest values in both precision and F1 score across nearly all models. This is likely due to the significant loss of informative data when a large portion of the majority class is removed.

On the other hand, Random Oversampling generally produces better results compared to SMOTE across all evaluation metrics, including accuracy, precision, recall, F1 score, and AUC. While SMOTE does enhance model performance in some instances, those models often exhibit relatively higher false negative rates—a critical issue in fraud detection, where missing fraudulent transactions can have serious consequences.

Among the oversampled models, XGBoost and Random Forest achieve comparable and notably high performance, indicating their robustness and effectiveness when combined with Random Oversampling.

Consequently, the most effective model and data balancing combinations in this study are:

- XGBoost with Random Oversampling
- Random Forest with Random Oversampling

These combinations strike an optimal balance between detecting fraudulent transactions and maintaining a low rate of false positives and false negatives.

References

- [1] Kaggle. Credit Card Fraud Detection. <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>
- [2] Gitub Page for this Report <https://github.com/Siddhesh2024/Credit-Card-Fraud-Detection-ML/tree/main>